从 GDN 到 LeapQuant：公式推导与 ATOM 实现对照

2026 年 10 月 22 日

本文对照三样东西推导：参考实现 `paper/gdn_verify.py`、ATOM 的 GDN kernel（提交 369abdf9b 加暂存区改动，文中行号均指该版本）、LeapQuant 的算法原理。所有恒等式都在 CPU 上用 fp64 做了数值验证，验证脚本和运行输出见附录。

目录

结论 **2**

0 记号 **2**

1 单步递推：`gdn_verify.py` 48–52 行 = `fused_recurrent.py` 141–155 行 **3**

2 展开成“检查点 + 记录”：ReplaySSM 的核心 **4**

3 不物化状态的读出：decode kernel 539–611 行 **5**

4 LeapQuant 的边界表示，以及免反量化的读出 **6**

5 量化器 `_lq_quantize`（247–287 行）的每一步 **7**

5.1 目标 . . . 8
5.2 给定方向时，最优的 $\tilde{u}$ 是投影 . . . 8
5.3 幂迭代 . . . 8
5.4 冷启动（prefill 写入时，`WARM=False`、`ITERS=8`） . . . 8
5.5 热启动（decode fold 时，`WARM=True`、`ITERS=0`） . . . 8
5.6 deflation 用舍入后的值 . . . 8
5.7 平滑 . . . 9
5.8 按列对称 int8 . . . 9

6 为什么误差不累积 **9**

6.1 窗口内，LeapQuant 等价于从 $\hat{S}_0$ 出发的精确递推 . . . 9
6.2 两条轨迹相减 . . . 10
6.3 跨窗口时，总误差是一个有界的几何级数 . . . 10
6.4 记录的 fp16 舍入 . . . 10

7 窗口节奏与游标 **10**

8 prefill 怎么接到同一套公式上 **11**

9 一个发现：每窗口只做一步热启动，补偿向量不够准 11

验证方式和局限 12

A 验证脚本 leapquant_derivation_check.py 12

B 消融脚本 leapquant_warm_ablation.py 18

结论

本文把整条推导从 gdn_verify.py 的参考递推一路推到 LeapQuant 的量化和误差分析，每一步都对上了 ATOM kernel 的行号，并在 CPU 上用 fp64 把每个恒等式数值验证了一遍。

ATOM 里的原生 decode、chunk prefill、ReplaySSM、LeapQuant，都可以写成同样三个公式的组合（原生 decode 相当于窗口长 1）：

1. **记录**：$u_t = \beta_t\,(v_t - \hat{k}_t\, e^{g_t} S_{t-1})$。它就是 gdn_verify.py 第 51 行括号里的 `beta * (v - v_old)`。
2. **回放**：$S_h = e^{C_h} S_0 + \sum_{j\le h} e^{C_h - C_j}\,\hat{k}_j^{\top} u_j$。
3. **读出**：$xS_h = e^{C_h}(xS_0) + \sum_{j\le h} e^{C_h - C_j}(x\cdot\hat{k}_j)\,u_j$。

LeapQuant 只是把检查点 S_0 换成 “int8 残差 + 平滑向量 + 4 个补偿 token” 的分解形式，让公式 3 里的 xS_0 不用反量化就能算。误差方面，窗口内的边界误差满足 $E_t = \alpha_t(I - \beta_t\hat{k}_t^{\top}\hat{k}_t)E_{t-1}$，范数只减不增。所以每 16 个 token 只在窗口边界引入一次量化误差。

fp64 验证中，所有恒等式的相对误差都在 1e-16 到 1e-14 之间，收缩不等式也成立。另外有一个新发现：fold 每个窗口只做一步热启动幂迭代（`WARM_ITERS=0`）。在合成数据上，这样拟合出的补偿向量不如最优解，输出误差高约 19%；多迭代一步就能补回大部分（见第 9 节）。

0 记号

ATOM 和 gdn_verify.py 都用**行向量**约定：

- 每个 v 头的状态 $S \in \mathbb{R}^{K\times V}$，行下标 i 是 key 维，列下标 j 是 value 维；
- $\hat{k}, \hat{q}$ 是 $1\times K$ 的行向量，v, u, o 是 $1\times V$ 的行向量；
- `einsum("hk,hkv->hv", k, S)` 就是 $\hat{k}S$。

论文用列向量写 $S^{\top}x$，和这里的 xS 互为转置，是同一个量。

符号	含义	gdn_verify.py	ATOM kernel
$\hat{k}$	$k/\|k\|$	第 43 行	fused_recurrent.py 135；leapquant.py 535
$\hat{q}$	$q/\|q\|\cdot K^{-1/2}$	第 42 行	fused_recurrent.py 134、137；leapquant.py 534、537
$g_t \le 0,\ \alpha_t = e^{g_t}$	衰减	第 32、44 行	fused_recurrent.py 140–141；leapquant.py 538–540
$\beta_t \in (0,1)$	写入强度	第 33 行	fused_recurrent.py 150；leapquant.py 608
$C_j = \sum_{i\le j} g_i$	窗口内累计 log 衰减	无	leapquant.py 550–552
分组	v 头 h 用 k 头 h//3	第 38–40 行	leapquant.py 505

1 单步递推：gdn_verify.py 48–52 行 = fused_recurrent.py 141–155 行

代码 1: paper/gdn_verify.py 第 47–52 行

```
47    for t in range(T):
48        S = alpha[t, :, None, None] * S # [48, 1, 1] * [48, 128, 128]
49        v_old = torch.einsum("hk,hkv->hv", k[t], S) # [48, 128] @ [48, 128, 128] -> [48, 128]
50        # [48, 128, 128] + [48, 128]  ([48, 1] * ([48, 128] - [48, 128]))
51        S = S + torch.einsum("hk,hv->hkv", k[t], beta[t, :, None] * (v[t] - v_old))
52        o[t] = torch.einsum("hk,hkv->hv", q[t], S)
```

代码 2: ATOM/atom/model_ops/fla_ops/fused_recurrent.py 第 139–155 行

```
139        if not IS_KDA:
140            b_g = tl.load(p_g).to(tl.float32)
141            b_h *= exp(b_g)
142        else:
143            b_gk = tl.load(p_gk).to(tl.float32)
144            b_h *= exp(b_gk[:, None])
145        # [BV]
146        b_v -= tl.sum(b_h * b_k[:, None], 0)
147        if IS_BETA_HEADWISE:
148            b_beta = tl.load(p_beta, mask=mask_v, other=0).to(tl.float32)
149        else:
150            b_beta = tl.load(p_beta).to(tl.float32)
151        b_v *= b_beta
152        # [BK, BV]
153        b_h += b_k[:, None] * b_v[None, :]
154        # [BV]
155        b_o = tl.sum(b_h * b_q[:, None], 0)
```

逐行对应如下：

- **衰减**：$S' = \alpha_t S_{t-1}$。对应 gdn_verify 第 48 行、kernel 第 141 行。
- **读旧值**：$v^{\text{old}} = \hat{k}_t S'$。对应第 49 行、第 146 行。
- **修正量**：$u_t = \beta_t (v_t - v^{\text{old}})$。对应第 51 行括号内、第 146 和 151 行。
- **写入**：$S_t = S' + \hat{k}_t^\top u_t$。对应第 51 行、第 153 行。
- **输出**：$o_t = \hat{q}_t S_t$。对应第 52 行、第 155 行。

合起来是：

$$S_t = S' + \hat{k}_t^\top \beta_t (v_t - \hat{k}_t S') = (I - \beta_t \hat{k}_t^\top \hat{k}_t)\, \alpha_t S_{t-1} + \beta_t \hat{k}_t^\top v_t$$

矩阵 $I - \beta \hat{k}^\top \hat{k}$ 的含义是：沿 $\hat{k}$ 方向把旧内容擦掉 β 的比例，其它方向不动。它的特征值是 1（$K-1$ 重）和 $1 - \beta\|\hat{k}\|^2$。因为 $\hat{k}$ 做过归一化、$\beta \in (0,1)$，它的谱范数正好是 1。第 6 节的误差分析全靠这一点。

整个递推里**只有 u_t 依赖旧状态**。u_t 一旦算出，这一步对状态的作用就只剩“整体乘一个标量，再加一个秩一外积”。同一个 u_t 在四处代码里有四个名字：

- gdn_verify.py 的 beta[t, :, None] * (v[t] - v_old)；

- `fused_recurrent.py` 第 151 行之后的 `b_v`；
- `chunk_delta_h.py` 的 `v_new`（第 215–231 行）；
- LeapQuant decode 的 `b_u`（第 609 行），也就是写进 `buf_u` 的记录。

另外，原生 kernel 每个 token 都要把整块状态写回（第 173 行在 token 循环里）。这是 decode 的带宽瓶颈。

2 展开成“检查点 + 记录”：ReplaySSM 的核心

从 $S_t = \alpha_t S_{t-1} + \hat{k}_t^\top u_t$ 开始逐步代入：

$$S_1 = \alpha_1 S_0 + \hat{k}_1^\top u_1, \qquad S_2 = \alpha_2\alpha_1 S_0 + \alpha_2 \hat{k}_1^\top u_1 + \hat{k}_2^\top u_2, \ \dots$$

$$S_h = \Big(\prod_{i=1}^{h} \alpha_i\Big) S_0 + \sum_{j=1}^{h} \Big(\prod_{i=j+1}^{h} \alpha_i\Big) \hat{k}_j^\top u_j = e^{C_h} S_0 + \sum_{j=1}^{h} e^{C_h - C_j}\, \hat{k}_j^\top u_j$$

用归纳法验证一步：$S_{h+1} = e^{g_{h+1}} S_h + \hat{k}_{h+1}^\top u_{h+1}$。把 $e^{g_{h+1}}$ 乘进去后，每一项的指数都加上 g_{h+1}；新记录的权重是 $e^{C_{h+1} - C_{h+1}} = 1$。

有三个细节：

1. **第 j 条记录自己的门不在它的权重里**。g_j 先衰减旧状态，然后才写入 $\hat{k}_j^\top u_j$，所以它的权重是 $e^{\sum_{i>j} g_i}$。
   - decode 里第 551 行 `b_w = exp(b_ctot - tl.cumsum(b_g1))`：`cumsum` 包含 j 自己，正好减掉。
   - `_lq_rebuild` 第 201–202 行先 `b_cj += b_gj`，再算 `exp(b_ctot - b_cj)`，是同一件事。
2. **为什么存 u 不存 v**。存 v 的话，求 u_j 需要 S_{j-1}，回放又变回串行。存 u 之后各项互相独立，回放就是一次缩放加一次 GEMM。
3. **不会溢出**。$g \le 0$，所有权重 $e^{C_h - C_j} \le 1$。

这个回放公式在 ATOM 里出现了三次：

- **`_lq_rebuild`**（`leapquant.py` 164–205 行）：
  - 166–170 行算 C_h 和 e^{C_h}；
  - 177–183 行算 $e^{C_h}\hat{S}_0$，衰减乘进 c 和补偿 key；
  - 193–204 行累加记录。
- **`_replay_records_kernel`**（`replayssm.py` 534–552 行）：从前缀缓存恢复时用。
- **chunk prefill 的块间递推**，窗口长 64。这里的 `b_g` 已经是块内前缀和 C_j：

代码 3: `ATOM/atom/model_ops/fla_ops/chunk_delta_h.py` 第 236–243 行

```
236             b_g_last = tl.load(g + bos * H + last_idx * H + i_h)
237             _p_g_0 = (i_t * BT) + tl.arange(0, BT)
238             b_g = tl.load(
239                 g + bos * H + i_h + _p_g_0 * (H), mask=(_p_g_0 < (T)), other=0.0
240             )
241             b_v = b_v * tl.where(m_t, exp(b_g_last - b_g), 0)[:, None]
242             b_g_last = exp(b_g_last)
243             b_h1 *= b_g_last
```

其中第 241 行乘的是 $e^{C_{\text{end}}-C_j}$，第 243 行是 $e^{C_{\text{end}}}S_0$，第 292 行 `b_h1 += tl.dot(b_k, b_v)` 是 $\sum_j \hat{k}_j^\top(\cdot)$。

chunk 里的 64 条记录是并行算出来的。把回放公式代入 u_j 的定义：

$$u_j = \beta_j v_j - \beta_j e^{C_j}\hat{k}_j S_0 - \sum_{i<j} \beta_j e^{C_j - C_i}(\hat{k}_j \cdot \hat{k}_i)\, u_i$$

按行堆成矩阵（第 j 行是 u_j）：

$$(I+L)\,U = \operatorname{diag}(\beta)V - \operatorname{diag}(\beta e^{C})\hat{K}S_0, \qquad L_{ji} = \beta_j e^{C_j - C_i}(\hat{k}_j \cdot \hat{k}_i) \;\; (i<j)$$

$$U = \underbrace{A\operatorname{diag}(\beta)V}_{u_{wy}} - \underbrace{A\operatorname{diag}(\beta e^{C})\hat{K}}_{w_{wy}}\, S_0, \qquad A = (I+L)^{-1}$$

这对应 `chunk_delta_h.py` 第 185–222 行：先算 `b_v = w @ h`，再算 `b_v = u - b_v`，结果存成 `v_new`。也就是说，decode 每步串行算一条记录，prefill 用三角求逆一次算出一块 64 条，之后两边用同一个回放公式。

验证：

- 回放公式与串行递推得到的 S_{16} 相差 2.8e-16；
- $u_{wy} - w_{wy}S_0$ 与串行记录相差 3.2e-16；
- chunk 形式的输出相差 5.7e-16。

3 不物化状态的读出：decode kernel 539–611 行

新 token 到来时，窗口里已有 h 条记录。它要用 $e^g S_h$ 两次（算 v^{old} 和算 o），但 kernel 从不把 S_h 拼出来。

第一步：把这一步的门推到向量上。因为 $\hat{k}\,(e^g S_h) = (e^g\hat{k})\,S_h$，令 $x_k = e^g\hat{k}$、$x_q = e^g\hat{q}$：

代码 4: `ATOM/atom/model_ops/fla_ops/leapquant.py` 第 538–542 行

```
538     b_gt = tl.load(g + bos * HV + i_hv).to(tl.float32)
539     # (e^g S_h)^T x == S_h^T (e^g x): this step's gate rides on the vectors.
540     b_eg = exp(b_gt)
541     b_xk = b_eg * b_k
542     b_xq = b_eg * b_q
```

第二步：任意行向量 x 对 S_h 的读出。把回放公式左乘 x，并利用 $x\,\hat{k}_j^\top u_j = (x\cdot\hat{k}_j)\,u_j$（$x\hat{k}_j^\top$ 是标量）：

$$xS_h = e^{C_h}\,(xS_0) + \sum_{j<h} \underbrace{e^{C_h - C_j}\,(x\cdot\hat{k}_j)}_{\text{标量}r_j}\, u_j$$

代码 5: `ATOM/atom/model_ops/fla_ops/leapquant.py` 第 571–574 行

```
571     b_rk = b_w * tl.sum(b_kr * b_xk[:, None], 0)
572     b_rq = b_w * tl.sum(b_kr * b_xq[:, None], 0)
573     b_sk = tl.sum(b_rk[:, None] * b_ur, 0)
574     b_sq = tl.sum(b_rq[:, None] * b_ur, 0)
```

`b_rk` 就是 r_j，`b_sk` 是记录部分的 $x_k S_h$。$e^{C_h}(xS_0)$ 这一项在第 580–597 行，见第 4 节。

第三步：修正量 $u = \beta\,(v - x_k S_h)$（第 609 行）。它就是 `gdn_verify.py` 的第 49、51 行：第 48 行已经把 S 乘过 α，所以 `v_old` $= \hat{k}(\alpha S_{t-1}) = (\alpha\hat{k})S_{t-1}$。

第四步：输出。

$$o = \hat{q} S_t = \hat{q}\,(e^{g} S_h + \hat{k}^{\top} u) = x_q S_h + (\hat{q} \cdot \hat{k})\, u$$

对应第 611 行 `b_sq + b_u * tl.sum(b_k * b_q)`。注意 $\hat{q}$ 带了 scale，$\hat{k}$ 没带。

第五步：追加记录。把 $(\hat{k}, u, g)$ 写到第 h 格（第 613–634 行），u 先截断到 fp16 的范围。

算量对比：拼出 S_h 要 hKV 次乘加，读出只要 $KV + h(K+V)$。

验证：在窗口的每个位置上，读出公式与串行递推的 u、o 分别相差 3.7e-16 和 6.5e-16。

4 LeapQuant 的边界表示，以及免反量化的读出

$$\hat{S}_0 = \mathrm{diag}(c)\, Z\, \mathrm{diag}(b) + \tilde{K}^{\top} \tilde{U}, \qquad \hat{S}_0[i,j] = c_i\, Z_{ij}\, b_j + \sum_{r=1}^{4} \tilde{k}_r[i]\, \tilde{u}_r[j]$$

每个头的存储：

- 128×128 的 int8：16384 B；
- b 和 c 共 256 个 fp32：1024 B；
- 4 个补偿 token，共 4×256 个 fp16：2048 B。

合计 19456 B，即 1.1875 B/元素，就是论文里的 1.19。`sc` 的布局是 b 在前、c 在后（`leapquant_buffer_shapes` 第 78–89 行）。窗口记录另占 $17 \times 256 \times 2 + 17 \times 4 = 8772$ B/头。

读出时换一下求和顺序：

$$(x\hat{S}_0)_j = \sum_i x_i c_i Z_{ij} b_j + \sum_r \Big(\sum_i x_i \tilde{k}_r[i] \Big) \tilde{u}_r[j] = b_j \big((x \odot c)\, Z \big)_j + \sum_r (x \cdot \tilde{k}_r)\, \tilde{u}_r[j]$$

再乘上 e^{C_h}：分别乘进 c（第 580 行）和补偿权重（第 596–597 行）。所以整块 int8 不用反量化成 fp32 矩阵，只需要一次“fp32 向量 × int8 矩阵”：

代码 6: `ATOM/atom/model_ops/fla_ops/leapquant.py` 第 580–597 行

```
580     b_c = tl.load(p_sc + V + o_k, mask=mask_k, other=0.0) * b_decay
581     b_b = tl.load(p_sc + o_v, mask=mask_v, other=0.0)
582     b_z = tl.load(
583         p_z + o_k[:, None] * V + o_v[None, :],
584         mask=mask_k[:, None] & mask_v[None, :],
585         other=0,
586     ).to(tl.float32)
587     b_sk += b_b * tl.sum(b_z * (b_c * b_xk)[:, None], 0)
588     b_sq += b_b * tl.sum(b_z * (b_c * b_xq)[:, None], 0)
589     p_row = p_comp + o_c[:, None] * (K + V)
590     b_ck = tl.load(
591         p_row + o_k[None, :], mask=mask_c[:, None] & mask_k[None, :], other=0.0
592     ).to(tl.float32)
593     b_cu = tl.load(
594         p_row + K + o_v[None, :], mask=mask_c[:, None] & mask_v[None, :], other=0.0
595     ).to(tl.float32)
596     b_sk += tl.sum((b_decay * tl.sum(b_ck * b_xk[None, :], 1))[:, None] * b_cu, 0)
597     b_sq += tl.sum((b_decay * tl.sum(b_ck * b_xq[None, :], 1))[:, None] * b_cu, 0)
```

补偿项 $e^{C_h}(x \cdot \tilde{k}_r)\,\tilde{u}_r$ 和记录项 $e^{C_h - C_j}(x \cdot \hat{k}_j)\,u_j$ 形状完全一样，相当于排在窗口最前面、门为 0 的“虚拟 token”。论文说补偿 token 复用同一条代码路径，指的就是这个。

需要稠密 $\hat{S}_0$ 的地方有两处：`_lq_boundary`（第 108–127 行，prefill 读入时用）和 `_lq_rebuild` 第 172–183 行（fold 重建时用）。

验证：$((c \odot x)Z) \odot b + \sum_r (x \cdot \tilde{k}_r)\tilde{u}_r$ 与 $x \cdot \mathrm{dequant}(\hat{S}_0)$ 相差 6.6e-16。

5 量化器 `_lq_quantize`（247–287 行）的每一步

代码 7: ATOM/atom/model_ops/fla_ops/leapquant.py 第 247–280 行

```
247     for r in tl.static_range(R):
248         if WARM:
249             b_uo = tl.load(p_comp + r * (K + V) + K + o_v, mask=mask_v, other=0.0)
250             b_kv = tl.sum(b_s * b_uo.to(tl.float32)[None, :], 1)
251             # The restart has to come out of the tile as `b_kv` does. Mixing
252             # in a separately loaded [K] vector (the old key) made the result
253             # vary from run to run with 2 warps on ROCm, barriers or not.
254             if tl.max(tl.abs(b_kv), 0) <= _TINY:
255                 b_kv = _lq_unit(tl.sqrt(tl.sum(b_s * b_s, 1)))
256                 for _ in tl.static_range(_COLD_ITERS):
257                     b_uv = tl.sum(b_s * b_kv[:, None], 0)
258                     b_kv = _lq_unit(tl.sum(b_s * b_uv[None, :], 1))
259         else:
260             # The row norms point at the outlier rows the tokens are for.
261             b_kv = tl.sqrt(tl.sum(b_s * b_s, 1))
262         b_kv = _lq_unit(b_kv)
263         for _ in tl.static_range(ITERS):
264             b_uv = tl.sum(b_s * b_kv[:, None], 0)
265             b_kv = _lq_unit(tl.sum(b_s * b_uv[None, :], 1))
266         b_uv = tl.sum(b_s * b_kv[:, None], 0)
267         b_kh = b_kv.to(tl.float16)
268         b_uh = tl.minimum(tl.maximum(b_uv, -_FP16_MAX), _FP16_MAX).to(tl.float16)
269         tl.store(p_comp + r * (K + V) + o_k, b_kh, mask=mask_k)
270         tl.store(p_comp + r * (K + V) + K + o_v, b_uh, mask=mask_v)
271         # Deflate by the rounded pair, so the residual is exactly what the
272         # stored compensators leave out.
273         b_s -= b_kh.to(tl.float32)[:, None] * b_uh.to(tl.float32)[None, :]
274
275     b_c = tl.maximum(tl.sqrt(tl.sum(tl.abs(b_s), 1) / V), SMOOTH_FLOOR)
276     b_s = b_s * (1.0 / b_c)[:, None]
277     b_amax = tl.max(tl.abs(b_s), 0)
278     b_b = tl.where(b_amax > _TINY, b_amax / 127.0, 1.0)
279     b_q = libdevice.rint(b_s * (1.0 / b_b)[None, :])
280     b_q = tl.minimum(tl.maximum(b_q, -127.0), 127.0)
```

5.1 目标

先找一个秩 4 的近似，即 $\min\|S - \tilde{K}^\top \tilde{U}\|_F$，最优解是前 4 个奇异分量。然后只量化剩下的残差。

5.2 给定方向时，最优的 $\tilde{u}$ 是投影

对单位向量 $\tilde{k}$：

$$\|S - \tilde{k}^\top \tilde{u}\|_F^2 = \|S\|_F^2 - 2\,\tilde{k}S\tilde{u}^\top + \|\tilde{u}\|^2 \;\Rightarrow\; \tilde{u}^* = \tilde{k}S, \quad \text{剩余能量} = \|S\|_F^2 - \|\tilde{k}S\|^2$$

所以要让 $\|\tilde{k}S\|^2 = \tilde{k}\,SS^\top\tilde{k}^\top$ 最大，$\tilde{k}$ 应取 $SS^\top$ 的主特征向量，也就是 S 的主左奇异向量。第 266 行算的就是 $\tilde{u} = \tilde{k}S$。

5.3 幂迭代

更新规则是 $\tilde{k} \leftarrow \mathrm{unit}\big((\tilde{k}S)S^\top\big)$，即第 263–265 行的两次归约。把 $\tilde{k}$ 按左奇异向量展开成 $\sum_i a_i e_i$，迭代 m 步后正比于 $\sum_i a_i \sigma_i^{2m} e_i$。偏离主方向的分量每步缩小 $(\sigma_2/\sigma_1)^2$ 倍，谱间隔越大收敛越快。

5.4 冷启动（prefill 写入时，`WARM=False`、`ITERS=8`）

初值取行范数 $\|S_{i,:}\|$（第 261 行）。如果 S 接近秩一的 $\tilde{k}_1^\top \tilde{u}_1$，行范数正好是 $|\tilde{k}_1[i]| \cdot \|\tilde{u}_1\|$，和主方向只差各分量的符号。在此基础上再迭代 8 步。

5.5 热启动（decode fold 时，`WARM=True`、`ITERS=0`）

初值是 $k = S\,\tilde{u}_{\text{old}}^\top = S\,S_{\text{old}}^\top \tilde{k}_{\text{old}}^\top$（第 249–250 行）。如果 $S \approx S_{\text{old}}$，这正是从上个窗口的 $\tilde{k}_{\text{old}}$ 出发走一步幂迭代。每个窗口走一步，跨窗口串起来，就是在一个缓慢漂移的矩阵上持续做幂迭代。`WARM_ITERS=0` 表示不再额外迭代，每个分量只花两次归约。

关于退化情况，有两点：

- 暂存区的修改（第 254–258 行）：当 $S\tilde{u}_{\text{old}}$ 接近 0 时，这个分量按冷启动处理，从行范数重新迭代 8 步。重启向量必须从 tile 归约得到，不能混入单独 load 的旧 key；否则在 ROCm 上用 2 个 warp 时结果不确定。
- `paper/leapquant_check.py` 里 `ref_quantize` 的第 96–100 行还是旧的回退逻辑（退化时直接用旧 key），和暂存区的 kernel 不一致。这只影响退化的 head，正常状态下结果相同。

5.6 deflation 用舍入后的值

第 273 行每次执行 $S \leftarrow S - \mathrm{fp16}(\tilde{k})^\top \mathrm{fp16}(\tilde{u})$，下一个分量在剩下的矩阵上找。热启动时第 r 个分量用的 $\tilde{u}_{\text{old},r}$，也来自上个窗口同样 deflate 过的矩阵，前后一致。

为什么要用 fp16 舍入后的值：反量化时加回去的是 fp16 的 $\tilde{K}^\top\tilde{U}$。记 $R := S_0 - \tilde{K}_{16}^\top \tilde{U}_{16}$，就有

$$\hat{S}_0 - S_0 = \mathrm{diag}(c)\,Z\,\mathrm{diag}(b) - R$$

误差只剩 int8 的舍入。如果用 fp32 值来 deflate，会多出一项 $\tilde{K}_{32}^\top\tilde{U}_{32} - \tilde{K}_{16}^\top\tilde{U}_{16}$，没有任何地方补偿它。

验证：$\hat{S}_0 - S_0$ 与 $\mathrm{diag}(c)(Z\,\mathrm{diag}(b) - R')$ 相差 2.1e-14。

5.7 平滑

第 275–276 行：$c_i = \max\left(\sqrt{\frac{1}{V}\sum_j |R_{ij}|}, 10^{-6}\right)$，$R' = \mathrm{diag}(c)^{-1}R$。记第 i 行的平均幅值为 m_i，平滑后变成 $m_i/\sqrt{m_i} = \sqrt{m_i}$。于是行与行之间的幅值比 ρ 变成 $\sqrt{\rho}$，在对数尺度上减半（验证误差 1.5e-16）。

为什么取平方根，以下是个人理解。元素误差的上界是 $c_i b_j/2$，而 b_j 由该列的 $\max_i |R_{ij}|/c_i$ 决定：

- 不平滑（$c \equiv 1$）时，列步长由最大的行决定，小行的格点很粗。
- 完全归一化（$c_i = m_i$）时，如果某个小行 i' 在第 j 列有尖峰，$|R_{i'j}|/m_{i'}$ 会很大，把整列步长撑大。大行 i 在这一列的误差就被放大 $m_i/m_{i'}$ 倍。
- 取 $c_i = \sqrt{m_i}$ 时，上面两种放大都只剩平方根。论文说这类似 SmoothQuant 的平方根平衡。

5.8 按列对称 int8

第 277–280 行：$b_j = \max_i |R'_{ij}|/127$，$Z = \mathrm{clamp}(\mathrm{rint}(R'/b), \pm 127)$。按构造有 $|R'_{ij}/b_j| \le 127$，clamp 只是保险。舍入误差满足 $|R'_{ij} - b_j Z_{ij}| \le b_j/2$，回到原坐标：

$$\left|\hat{S}_0 - S_0\right|_{ij} \le \tfrac{1}{2}\, c_i\, b_j$$

相应地，读出误差满足 $|x(\hat{S}_0 - S_0)|_j \le \frac{b_j}{2}\sum_i |x_i|\, c_i$。

验证：$\max_{ij} |\hat{S}_0 - S_0|_{ij}/(c_i b_j/2) = 0.999995$，说明这个界是紧的。

还有两处数值保护：

- 全零列取 $b = 1$、$Z = 0$（第 278 行）；
- `_lq_unit`（第 209–219 行）先按最大值缩放再归一化，避免残差很小时平方和在 fp32 下下溢。这就是实验结论里那次 NaN 的修复。

单次重建误差（合成状态）：LeapQuant 3.70e-3，普通按列 INT8 9.34e-3，BF16 1.65e-3。只看一次量化，int8 的 LeapQuant 还不如 BF16；它的优势来自下一节的“每个窗口只量化一次”。

6 为什么误差不累积

6.1 窗口内，LeapQuant 等价于从 $\hat{S}_0$ 出发的精确递推

定义等效状态

$$\hat{S}_t = e^{C_t}\hat{S}_0 + \sum_{j \le t} e^{C_t - C_j}\hat{k}_j^{\top}\hat{u}_j$$

其中 $\hat{u}_j$ 是 kernel 实际写入的记录。由第 3、4 节，kernel 的读出精确等于 $x\hat{S}_t$，所以

$$\hat{u}_t = \beta_t\,(v_t - \hat{k}_t\, e^{g_t}\hat{S}_{t-1}), \qquad \hat{S}_t = \alpha_t \hat{S}_{t-1} + \hat{k}_t^{\top}\hat{u}_t$$

这和第 1 节的递推完全一样，只是起点换成了 $\hat{S}_0$（不计记录的 fp16 舍入）。`leapquant_check.py` B 部分的 shadow 正是基于这一点：每次 flush 后换成 kernel 的边界，再做精确的 fp32 步进。

验证：分解形式的 LeapQuant decode 与从 $\hat{S}_0$ 出发的稠密递推相差 7.1e-16。

6.2 两条轨迹相减

与状态无关的 $\beta_t \hat{k}_t^\top v_t$ 相互抵消：

$$E_t = \hat{S}_t - S_t = \alpha_t \left(I - \beta_t \hat{k}_t^\top \hat{k}_t\right) E_{t-1}, \qquad \|E_t\|_F \le \alpha_t \|E_{t-1}\|_F$$

不等式用的是第 1 节的结论：$I - \beta \hat{k}^\top \hat{k}$ 的谱范数为 1。所以窗口内不产生新误差，旧误差随衰减变小，沿 $\hat{k}$ 方向还会被额外擦掉 β 的比例。

验证：E_t 与右边的预测相差 6.0e-14；$\|E_t\|/(\alpha_t\|E_{t-1}\|)$ 的最大值是 0.9997。

这和论文附录 B.1 的 $E_t = \mathrm{Diag}(\alpha_t)E_{t-1} - k_t(E_{t-1}^\top \beta_t)^\top + \varepsilon_t$ 是同一个式子（论文用列向量写）。区别只在于：逐步量化每一步都有新的 ε_t，LeapQuant 只在窗口边界那一步有。

6.3 跨窗口时，总误差是一个有界的几何级数

第 m 个窗口边界引入误差 ε_m，之后一路被收缩。逐窗口量化时，每次注入后到下次注入前衰减 α^{16}；逐步量化时只衰减 α。分两种情况：

- **误差系统性同向**（论文所说的“小增量被舍入系统性吃掉”）：逐窗口的稳态误差约为 $\varepsilon/(1-\alpha^{16})$，逐步约为 $\varepsilon/(1-\alpha)$。$\alpha \to 1$ 时两者之比趋于 16。
- **误差随机独立**：按平方和累加，比值趋于 $\sqrt{16} = 4$。

无论哪种情况，误差都不会随长度无限增长，大约 $1/(1-\alpha^{16})$ 个窗口后饱和。论文用 MSE（平方）衡量，降幅是 39–100 倍，换算成范数是 6–10 倍，正好落在 4 到 16 之间。

用同一个量化器、只改量化频率做实验（256 个 token，分四段取平均）：

- LeapQuant 每窗口一次：7.3e-4 → 8.5e-4；每 token 一次：2.6e-3 → 3.3e-3，差约 3.7 倍。
- 普通 INT8 每窗口一次：1.25e-3 → 1.44e-3；每 token 一次：4.8e-3 → 5.8e-3。

这组数据 α 的中位数约 0.94。按上面的公式，随机情形预测约 2.7 倍，完全同向预测约 11 倍；实测的 3.7 倍更接近随机情形。慢衰减时（$\alpha \approx 0.99$），每窗口量化的误差在前两段上涨、后两段持平（1.7e-3 → 2.1e-3 → 2.3e-3 → 2.2e-3），也符合“约 8 个窗口后饱和”的预测。在 GPU 上用 kernel 跑的 `leapquant_check.py` B 部分也是同样的规律（见 `LeapQuant_ATOM_实验结论.md`）：LeapQuant 5.1e-4 → 5.3e-4，每步普通 INT8 3.6e-3 → 4.0e-3。

6.4 记录的 fp16 舍入

$\hat{u}$ 和 $\hat{k}$ 以 fp16 存储，每个 token 确实会引入一点误差。但它的量级是单个增量 $\hat{k}^\top \hat{u}$ 的 2^{-11}，而不是整块状态的；重建和累积都在 fp32 里做（`_lq_rebuild`）。这就是“高精度累积、低频舍入”。

7 窗口节奏与游标

三处判断用的是同一个条件：

- **commit**（`replayssm.py` 第 431–438 行）：$\mathrm{pos}' = (\mathrm{pos} + 2 > \mathrm{CAP}\ ?\ 0 : \mathrm{pos}) + 1$（$T = 1$，接受 1 个 token）；pos 为 -1 时直接变成 0。
- **decode**（`leapquant.py` 第 516–517 行）：$h = \max(\mathrm{pos}, 0)$；若 $h + 2 > \mathrm{CAP}$，则 $h = 0$。
- **fold**（第 348–350 行）：只在 $h + 2 > \mathrm{CAP}$ 时执行，用的就是这 h 条记录。

CAP = 17 时的时间线：

1. prefill 之后 pos $= -1$，commit 后变成 0；

2. decode 写入记录 0，commit 后 pos = 1；如此重复，直到 pos = 16（记录 0–15，正好 16 个 token）；
3. 下一步 $16+2>17$：fold 用这 16 条记录重建 $\hat{S}_{16}$ 并重新量化；decode 看到 $h=0$，写入新的记录 0；commit 后 $\text{pos}=0+1=1$。

所以 CAP 要取“窗口长度 + 1”（`gdn_attn.py` 第 261 行）。

`replayssm_records_in_state`（第 490–492 行）算的 $n=(\text{pos}+2>\text{CAP}\ ?\ 0:\text{pos})+1$，就是上次 decode 读到的 h，再加上它自己追加的 1 条。从前缀缓存恢复时，$S=\text{replay}(\hat{S}_0,\ \text{前 } n \text{ 条})=\hat{S}_n$，由第 2 节的回放公式精确成立（`attention_gdn.py` 第 490–501 行）。

8 prefill 怎么接到同一套公式上

prefill 依次做四件事：

1. 用 `_lq_boundary` 把 $\hat{S}_0$ 读成稠密矩阵；
2. 如果是从 decode 中途的 checkpoint 恢复，再回放 n 条记录；
3. 从这个初始状态跑 chunk prefill，它就是精确递推的分块写法（第 2 节）；
4. 末状态用冷启动量化（8 步幂迭代），游标置为 -1。

读入边界时带进来的量化误差，在 prefill 里按 6.2 节的同一个式子收缩。

prefill 中途的 checkpoint 就是每块开头的状态 h_c：`chunk_delta_h.py` 第 145–176 行把它们存下来，再由 `leapquant_write_checkpoints` 量化，同时 `write_state_checkpoints` 传入 `copy_ssm=False`。不过 `LeapQuant_ATOM_实验结论.md` 中记录，ATOM 目前的调度走不到这条路径。

9 一个发现：每窗口只做一步热启动，补偿向量不够准

5.5 节的热启动是在“跟踪”主子空间，并不是每次都解到收敛。在合成数据上比较 fold 时几种迭代设置。补偿残差定义为 $\|S-\tilde{K}^{\top}\tilde{U}\|/\|S\|$，越小越好；输出误差取四段的平均值。

fold 设置	补偿残差（快 / 慢衰减）	输出误差（快衰减）	输出误差（慢衰减）
热启动 0 步（ATOM 现状）	0.555 / 0.736	5.37e-4	2.07e-3
热启动 1 步	0.474 / 0.709	4.69e-4	2.01e-3
热启动 2 步	0.458 / 0.701	4.55e-4	2.00e-3
冷启动 8 步	0.448 / 0.695	4.50e-4	1.97e-3

在另一组数据上，冷启动 8 步的残差是 0.5045，SVD 最优解是 0.5029，基本就是最优解。

快衰减时 $\alpha^{16}\approx 0.39$，一个窗口就把状态更新掉约 60%。再加上合成状态的谱比较平（前 1 个和前 4 个奇异值只占 42% 和 75% 的能量），一步幂迭代跟不上，输出误差比冷启动 8 步高约 19%；慢衰减时只高约 5%。多迭代一步，差距就缩到 4% 和 2%。

真实状态低秩得多：按 `LeapQuant_论文解读.md` 中从论文图 5 读出的数值，前 1 个奇异值约占 81% 的能量，前 4 个约占 96%。所以真实状态上的差距应该更小，但尚未在真实状态上测过。`leapquant_fold` 有 `iters` 参数，改成 1 会让每个分量多两次归约。$B=512$ 时，fold 占 GDN 总耗时 5.20 ms 中的 2.25 ms，值不值得改需要实测。

验证方式和局限

- 验证脚本有两个：`leapquant_derivation_check.py` 覆盖第 1–8 节的恒等式和误差实验，`leapquant_warm_ablation.py` 是第 9 节的消融。两者都是 CPU 上 fp64 的 torch 镜像，输入生成方式照搬 `leapquant_check.py` 的 `Source`。脚本全文和运行输出见附录 A、B（原文件放在 `/tmp` 下，可能会被清理）。
- 这两个脚本验证的是**数学推导**，不是 Triton kernel 本身。kernel 与镜像是否一致，靠的是 `leapquant_check.py` 的 A、B 部分。这两部分在暂存区的热启动修改之后还没重跑（整理时 GPU 被占满）。
- 如果想在 `gdn_verify.py` 的真实权重上也看到第 6 节的效果，可以加一条 shadow 路径：每 16 个 token 把 S 换成 `dequant(quantize(S))`，再和 `ref_recurrence` 的输出比较。

A 验证脚本 `leapquant_derivation_check.py`

覆盖第 1–8 节的恒等式与误差实验。在 CPU 上用 fp64 运行，约 3 秒：`CUDA_VISIBLE_DEVICES="" python leapquant_derivation_check.py`。

代码 8: `leapquant_derivation_check.py`

```
"""fp64 CPU checks of the GDN -> ReplaySSM -> LeapQuant derivation.

Each batch row is one v-head; inputs mimic paper/leapquant_check.py's Source
(per-channel scales plus a shared direction per head, so the state has the
outlier rows/columns and low-rank energy real ones have).
"""
import torch
import torch.nn.functional as F

torch.manual_seed(0)
torch.set_default_dtype(torch.float64)
NH, K, V, P, R = 8, 128, 128, 16, 4
SCALE = K**-0.5
TINY = 1e-30


class Source:
    def __init__(self):
        self.ks = (torch.randn(NH, K) * 0.8).exp()
        self.kb = torch.randn(NH, K) * 2
        self.vs = (torch.randn(NH, V) * 0.8).exp()
        self.vb = torch.randn(NH, V)
        self.gb = torch.randn(NH) * 0.5 - 3

    def step(self):
        q = torch.randn(NH, K) * self.ks + self.kb
        k = torch.randn(NH, K) * self.ks + self.kb
        v = torch.randn(NH, V) * self.vs + self.vb
        g = -F.softplus(torch.randn(NH) * 0.5 + self.gb)
        beta = torch.sigmoid(torch.randn(NH))
```

```
        q = q / torch.sqrt((q * q).sum(-1, keepdim=True) + 1e-6) * SCALE
        k = k / torch.sqrt((k * k).sum(-1, keepdim=True) + 1e-6)
        return q, k, v, g, beta


def step(S, q, k, v, g, beta):
    """gdn_verify.py ref_recurrence, lines 48-52."""
    S = g.exp()[:, None, None] * S
    v_old = torch.einsum("hk,hkv->hv", k, S)
    u = beta[:, None] * (v - v_old)
    S = S + torch.einsum("hk,hv->hkv", k, u)
    return S, torch.einsum("hk,hkv->hv", q, S), u


def rel(a, b):
    return ((a - b).norm() / b.norm()).item()


def unit(x):
    m = x.abs().amax(-1, keepdim=True)
    x = torch.where(m > TINY, x / m.clamp_min(TINY), 0.0)
    return x / x.norm(dim=-1, keepdim=True).clamp_min(1.0)


def power(S, kv, iters):
    kv = unit(kv)
    for _ in range(iters):
        uv = torch.einsum("hkv,hk->hv", S, kv)
        kv = unit(torch.einsum("hkv,hv->hk", S, uv))
    return kv


def quantize(S, iters, warm=None):
    """fp64 mirror of `_lq_quantize` (staged version, incl. the cold restart)."""
    S = S.clone()
    comp = torch.zeros(NH, R, K + V)
    for r in range(R):
        if warm is None:
            kv = S.square().sum(-1).sqrt()
        else:
            kv = torch.einsum("hkv,hv->hk", S, warm[:, r, K:])
            dead = kv.abs().amax(-1, keepdim=True) <= TINY
            if dead.any():
                kv = torch.where(dead, power(S, S.square().sum(-1).sqrt(), 8), kv)
        kv = power(S, kv, iters)
        uv = torch.einsum("hkv,hk->hv", S, kv)
        kh = kv.half().double()
        uh = uv.clamp(-65504, 65504).half().double()
        comp[:, r, :K], comp[:, r, K:] = kh, uh
        S = S - kh[:, :, None] * uh[:, None, :]
```

```
    c = S.abs().mean(-1).sqrt().clamp_min(1e-6)
    Rs = S / c[:, :, None]
    amax = Rs.abs().amax(-2)
    b = torch.where(amax > TINY, amax / 127, 1.0)
    Z = (Rs / b[:, None, :]).round().clamp(-127, 127)
    return dict(Z=Z, b=b, c=c, comp=comp, R=S, Rs=Rs)


def low_rank(comp):
    return torch.einsum("hrk,hrv->hkv", comp[:, :, :K], comp[:, :, K:])


def dequant(qs):
    return qs["c"][:, :, None] * qs["Z"] * qs["b"][:, None, :] + low_rank(qs["comp"])


def int8_per_channel(S):
    b = S.abs().amax(-2, keepdim=True).clamp_min(TINY) / 127
    return (S / b).round().clamp(-127, 127) * b


def outlier_ratio(X):
    m = X.abs().amax((-2, -1)) / X.abs().median(-1).values.median(-1).values
    return m.median().item()


src = Source()
S = torch.zeros(NH, K, V)
for _ in range(512):
    S, _, _ = step(S, *src.step())
S0 = S.clone()
toks = [src.step() for _ in range(P)]

# [1] records + replay == recurrence
S = S0.clone()
recs, outs = [], []
for q, k, v, g, beta in toks:
    S, o, u = step(S, q, k, v, g, beta)
    recs.append((k, u, g))
    outs.append(o)
S_P = S
Kr = torch.stack([r[0] for r in recs], 1)
Ur = torch.stack([r[1] for r in recs], 1)
Gr = torch.stack([r[2] for r in recs], 1)
C = Gr.cumsum(1)
S_rep = C[:, -1].exp()[:, None, None] * S0 + torch.einsum(
    "hj,hjk,hjv->hkv", (C[:, -1:] - C).exp(), Kr, Ur
)
print(f"[1] replay formula vs serial recurrence, S_16      rel {rel(S_rep, S_P):.1e}")

```

```
# [2] chunk (WY/UT) form: v_new == records u, and chunk outputs
Br = torch.stack([t[4] for t in toks], 1)
Vr = torch.stack([t[2] for t in toks], 1)
Qr = torch.stack([t[0] for t in toks], 1)
D = C[:, :, None] - C[:, None, :]
L = torch.tril(Br[:, :, None] * D.exp() * torch.einsum("hik,hjk->hij", Kr, Kr), -1)
A = torch.linalg.inv(torch.eye(P) + L)
u_wy = A @ (Br[..., None] * Vr)
w_wy = A @ ((Br * C.exp())[..., None] * Kr)
v_new = u_wy - w_wy @ S0
M = torch.tril(D.exp() * torch.einsum("hik,hjk->hij", Qr, Kr))
o_chunk = C.exp()[..., None] * torch.einsum("hjk,hkv->hjv", Qr, S0) + M @ v_new
print(f"[2] chunk v_new = u_wy - w_wy S0 vs records u        rel {rel(v_new, Ur):.1e}")
print(f"[2] chunk o = (q e^C) S0 + mask(QK^T e^(Ci-Cj)) v_new rel {rel(o_chunk, torch.stack(
    outs, 1)):.1e}")


# [3] decode readout from vectors only (leapquant.py 539-611), dense boundary
def readout_dense(x, S0, Kh, Uh, g1):
    ctot = g1.sum(1)
    w = (ctot[:, None] - g1.cumsum(1)).exp()
    rec = torch.einsum("hj,hjv->hv", w * torch.einsum("hjk,hk->hj", Kh, x), Uh)
    return ctot.exp()[:, None] * torch.einsum("hk,hkv->hv", x, S0) + rec


wu = wo = 0.0
for h, (q, k, v, g, beta) in enumerate(toks):
    xk, xq = g.exp()[:, None] * k, g.exp()[:, None] * q
    sk = readout_dense(xk, S0, Kr[:, :h], Ur[:, :h], Gr[:, :h])
    sq = readout_dense(xq, S0, Kr[:, :h], Ur[:, :h], Gr[:, :h])
    u = beta[:, None] * (v - sk)
    o = sq + u * (k * q).sum(-1, keepdim=True)
    wu, wo = max(wu, rel(u, Ur[:, h])), max(wo, rel(o, outs[h]))
print(f"[3] vector readout vs recurrence, worst over h      u {wu:.1e}  o {wo:.1e}")

# [4] quantiser algebra
qs = quantize(S0, iters=8)
S0h = dequant(qs)
x = torch.randn(NH, K)
fact = ((qs["c"] * x)[:, None, :] @ qs["Z"]).squeeze(1) * qs["b"] + torch.einsum(
    "hr,hrv->hv", torch.einsum("hrk,hk->hr", qs["comp"][:, :, :K], x), qs["comp"][:, :, K:]
)
print(f"[4a] ((c*x)Z)*b + sum (x.k~)u~  vs  x @ dequant      rel {rel(fact, torch.einsum('hk,
    hkv->hv', x, S0h)):.1e}")
bound = 0.5 * qs["c"][:, :, None] * qs["b"][:, None, :]
print(f"[4b] max |S^-S| / (c_i b_j / 2)                      {((S0h - S0).abs() / bound).max
    ().item():.6f}")
keep = qs["c"] > 1e-6
lhs = qs["Rs"].abs().mean(-1)[keep]
rhs = qs["R"].abs().mean(-1).sqrt()[keep]
```

```
print(f"[4c] mean_j|R'_ij| vs sqrt(mean_j|R_ij|)              rel {rel(lhs, rhs):.1e}")
err_int8 = qs["c"][:, :, None] * (qs["Z"] * qs["b"][:, None, :] - qs["Rs"])
print(f"[4d] S^-S vs c*(Z*b - R') (only int8 rounding left)   rel {rel(S0h - S0, err_int8):.1e}")
print(f"     outlier max/median: state {outlier_ratio(S0):.0f}x -> residual "
      f"{outlier_ratio(qs['R']):.0f}x -> smoothed {outlier_ratio(qs['Rs']):.1f}x")
print(f"     one-shot rel error: LeapQuant {rel(S0h, S0):.2e}  plain INT8/col "
      f"{rel(int8_per_channel(S0), S0):.2e}  BF16 {rel(S0.bfloat16().double(), S0):.2e}")

# [5] error propagation inside a window, and LeapQuant decode == exact steps from S^_0
E = S0h - S0
S, Sh = S0.clone(), S0h.clone()
wrec = wratio = wshadow = 0.0
rk_, ru_, rg_ = [], [], []
for h, (q, k, v, g, beta) in enumerate(toks):
    xk, xq = g.exp()[:, None] * k, g.exp()[:, None] * q
    g1 = torch.stack(rg_, 1) if h else torch.zeros(NH, 0)
    Kh = torch.stack(rk_, 1) if h else torch.zeros(NH, 0, K)
    Uh = torch.stack(ru_, 1) if h else torch.zeros(NH, 0, V)
    ctot = g1.sum(1)
    w, decay = (ctot[:, None] - g1.cumsum(1)).exp(), ctot.exp()

    def readout(x):
        rec = torch.einsum("hj,hjv->hv", w * torch.einsum("hjk,hk->hj", Kh, x), Uh)
        bnd = ((qs["c"] * decay[:, None] * x)[:, None, :] @ qs["Z"]).squeeze(1) * qs["b"]
        cw = decay[:, None] * torch.einsum("hrk,hk->hr", qs["comp"][:, :, :K], x)
        return rec + bnd + torch.einsum("hr,hrv->hv", cw, qs["comp"][:, :, K:])

    u = beta[:, None] * (v - readout(xk))
    o = readout(xq) + u * (k * q).sum(-1, keepdim=True)
    rk_.append(k), ru_.append(u), rg_.append(g)

    S, _, _ = step(S, q, k, v, g, beta)
    Sh, oh, _ = step(Sh, q, k, v, g, beta)
    En = Sh - S
    pred = g.exp()[:, None, None] * (E - beta[:, None, None] * torch.einsum("hk,hj,hjv->hkv",
     k, k, E))
    wrec = max(wrec, rel(En, pred))
    ratio = En.flatten(1).norm(dim=1) / (g.exp() * E.flatten(1).norm(dim=1))
    wratio = max(wratio, ratio.max().item())
    wshadow = max(wshadow, rel(o, oh))
    E = En
print(f"[5a] E_t vs a_t (I - b_t k k^T) E_(t-1)               rel {wrec:.1e}")
print(f"[5b] max ||E_t||_F / (a_t ||E_(t-1)||_F)             {wratio:.6f}")
print(f"[5c] LeapQuant decode vs exact steps from S^_0       rel {wshadow:.1e}")


# [6] warm start (one chained power step per window) vs cold-8 vs optimal rank-4
def resid(S, comp):
    return ((S - low_rank(comp)).flatten(1).norm(dim=1) / S.flatten(1).norm(dim=1))
```

```

sv = torch.linalg.svdvals(S0)
e = sv.square() / sv.square().sum(1, keepdim=True)
print(f"[6] energy in top-1 / top-4 singular values (median)  "
      f"{e[:, 0].median().item():.1%} / {e[:, :4].sum(1).median().item():.1%}")
Sb, comp = dequant(qs), qs["comp"]
rw = rc = ro = 0.0
W = 32
for _ in range(W):
    for _ in range(P):
        Sb, _, _ = step(Sb, *src.step())
    sv = torch.linalg.svdvals(Sb)
    ro += (sv[:, R:].square().sum(1).sqrt() / sv.square().sum(1).sqrt()).mean().item()
    rc += resid(Sb, quantize(Sb, 8)["comp"]).mean().item()
    qw = quantize(Sb, 0, warm=comp)
    rw += resid(Sb, qw["comp"]).mean().item()
    Sb, comp = dequant(qw), qw["comp"]
print(f"[6] ||S - K~^T U~|| / ||S||, mean over {W} windows: warm(0 it) {rw / W:.4f}  "
      f"cold(8 it) {rc / W:.4f}  SVD optimum {ro / W:.4f}")

# [7] same quantiser, once per window vs once per token
steps = 256
ex, pw, ps, iw, is_ = (S0.clone() for _ in range(5))
qa = quantize(S0, 8)
pw, ps = dequant(qa), dequant(qa)
cw = cs = qa["comp"]
iw = is_ = int8_per_channel(S0)
err = {n: [] for n in ("LeapQuant/window", "LeapQuant/token", "INT8/window", "INT8/token")}
for t in range(steps):
    tok = src.step()
    if t and t % P == 0:
        qq = quantize(pw, 0, warm=cw)
        pw, cw = dequant(qq), qq["comp"]
        iw = int8_per_channel(iw)
    ex, o_ex, _ = step(ex, *tok)
    pw, o_pw, _ = step(pw, *tok)
    ps, o_ps, _ = step(ps, *tok)
    qq = quantize(ps, 0, warm=cs)
    ps, cs = dequant(qq), qq["comp"]
    iw, o_iw, _ = step(iw, *tok)
    is_, o_is, _ = step(is_, *tok)
    is_ = int8_per_channel(is_)
    for n, o in zip(err, (o_pw, o_ps, o_iw, o_is)):
        err[n].append(rel(o, o_ex))
print(f"[7] output rel error vs exact, mean per quarter of {steps} tokens:")
for n, e in err.items():
    q4 = steps // 4
    print(f"    {n:17s} " + "  ".join(f"{sum(e[i:i + q4]) / q4:.2e}" for i in range(0, steps,
     q4)))
```

运行输出如下。其中 outlier 一行是全矩阵的 max/median 比值：去掉低秩部分后中位数下降得比最大值更多，这个指标在合成数据上不单调，正文没有使用。

```
[1] replay formula vs serial recurrence, S_16      rel 2.8e-16
[2] chunk v_new = u_wy - w_wy S0 vs records u        rel 3.2e-16
[2] chunk o = (q e^C) S0 + mask(QK^T e^(Ci-Cj)) v_new rel 5.7e-16
[3] vector readout vs recurrence, worst over h      u 3.7e-16  o 6.5e-16
[4a] ((c*x)Z)*b + sum (x.k~)u~  vs  x @ dequant      rel 6.6e-16
[4b] max |S^-S| / (c_i b_j / 2)                       0.999995
[4c] mean_j|R'_ij| vs sqrt(mean_j|R_ij|)              rel 1.5e-16
[4d] S^-S vs c*(Z*b - R') (only int8 rounding left)   rel 2.1e-14
     outlier max/median: state 43x -> residual 59x -> smoothed 31.4x
     one-shot rel error: LeapQuant 3.70e-03  plain INT8/col 9.34e-03  BF16 1.65e-03
[5a] E_t vs a_t (I - b_t k k^T) E_(t-1)               rel 6.0e-14
[5b] max ||E_t||_F / (a_t ||E_(t-1)||_F)              0.999709
[5c] LeapQuant decode vs exact steps from S^_0       rel 7.1e-16
[6] energy in top-1 / top-4 singular values (median)  40.2% / 71.4%
[6] ||S - K~^T U~|| / ||S||, mean over 32 windows: warm(0 it) 0.6006  cold(8 it) 0.5045  SVD optimum 0.5029
[7] output rel error vs exact, mean per quarter of 256 tokens:
    LeapQuant/window  7.27e-04  8.12e-04  8.23e-04  8.47e-04
    LeapQuant/token   2.64e-03  3.09e-03  3.18e-03  3.25e-03
    INT8/window       1.25e-03  1.32e-03  1.41e-03  1.44e-03
    INT8/token        4.79e-03  5.51e-03  5.76e-03  5.84e-03
```

B 消融脚本 leapquant_warm_ablation.py

第 9 节的消融。它复用附录 A 脚本中 `src = Source()` 之前的全部定义（通过 `exec` 读入）。

代码 9: leapquant_warm_ablation.py

```python
"""Does one chained power step per window (WARM_ITERS=0) cost output accuracy?

Same fp64 mirror as leapquant_derivation_check.py; two decay regimes.
"""
import sys

import torch

sys.argv = ["x"]
exec(open("/tmp/leapquant_derivation_check.py").read().split("src = Source()")[0])


def run(g_shift, label, steps=256):
    torch.manual_seed(1)
    src = Source()
    src.gb = src.gb + g_shift
    S = torch.zeros(NH, K, V)
    for _ in range(512):
        S, _, _ = step(S, *src.step())
    toks = [src.step() for _ in range(steps)]
    a = torch.stack([t[3] for t in toks]).exp()
    sv = torch.linalg.svdvals(S)
    e = sv.square() / sv.square().sum(1, keepdim=True)
```

```
    print(f"== {label}: alpha median {a.median().item():.3f}, alpha^16 {a.median().item() ** 16:.2f}, "
          f"top-1/top-4 energy {e[:, 0].median().item():.0%}/{e[:, :4].sum(1).median().item():.0%}")
    variants = {"warm 0 it (ATOM)": (0, True), "warm 1 it": (1, True),
                "warm 2 it": (2, True), "cold 8 it": (8, False)}
    q0 = quantize(S, 8)
    for name, (iters, warm) in variants.items():
        ex, st, comp = S.clone(), dequant(q0), q0["comp"]
        errs, res = [], []
        for t, tok in enumerate(toks):
            if t and t % P == 0:
                qq = quantize(st, iters, warm=comp if warm else None)
                res.append(resid(st, qq["comp"]).mean().item())
                st, comp = dequant(qq), qq["comp"]
            ex, o_ex, _ = step(ex, *tok)
            st, o, _ = step(st, *tok)
            errs.append(rel(o, o_ex))
        q4 = steps // 4
        print(f"  {name:17s} comp resid {sum(res) / len(res):.3f}  out err "
              + "  ".join(f"{sum(errs[i:i + q4]) / q4:.2e}" for i in range(0, steps, q4)))


def resid(S, comp):
    return ((S - low_rank(comp)).flatten(1).norm(dim=1) / S.flatten(1).norm(dim=1))


run(0.0, "fast decay (as above)")
run(-2.0, "slow decay")
```

运行输出：

```
== fast decay (as above): alpha median 0.942, alpha^16 0.39, top-1/top-4 energy 42%/75%
  warm 0 it (ATOM)  comp resid 0.555  out err 4.94e-04  5.51e-04  5.80e-04  5.21e-04
  warm 1 it         comp resid 0.474  out err 4.51e-04  4.69e-04  4.95e-04  4.62e-04
  warm 2 it         comp resid 0.458  out err 4.38e-04  4.58e-04  4.72e-04  4.51e-04
  cold 8 it         comp resid 0.448  out err 4.36e-04  4.46e-04  4.75e-04  4.43e-04
== slow decay: alpha median 0.992, alpha^16 0.88, top-1/top-4 energy 20%/48%
  warm 0 it (ATOM)  comp resid 0.736  out err 1.72e-03  2.10e-03  2.26e-03  2.21e-03
  warm 1 it         comp resid 0.709  out err 1.66e-03  2.03e-03  2.18e-03  2.15e-03
  warm 2 it         comp resid 0.701  out err 1.66e-03  2.05e-03  2.17e-03  2.12e-03
  cold 8 it         comp resid 0.695  out err 1.62e-03  1.99e-03  2.18e-03  2.08e-03
```